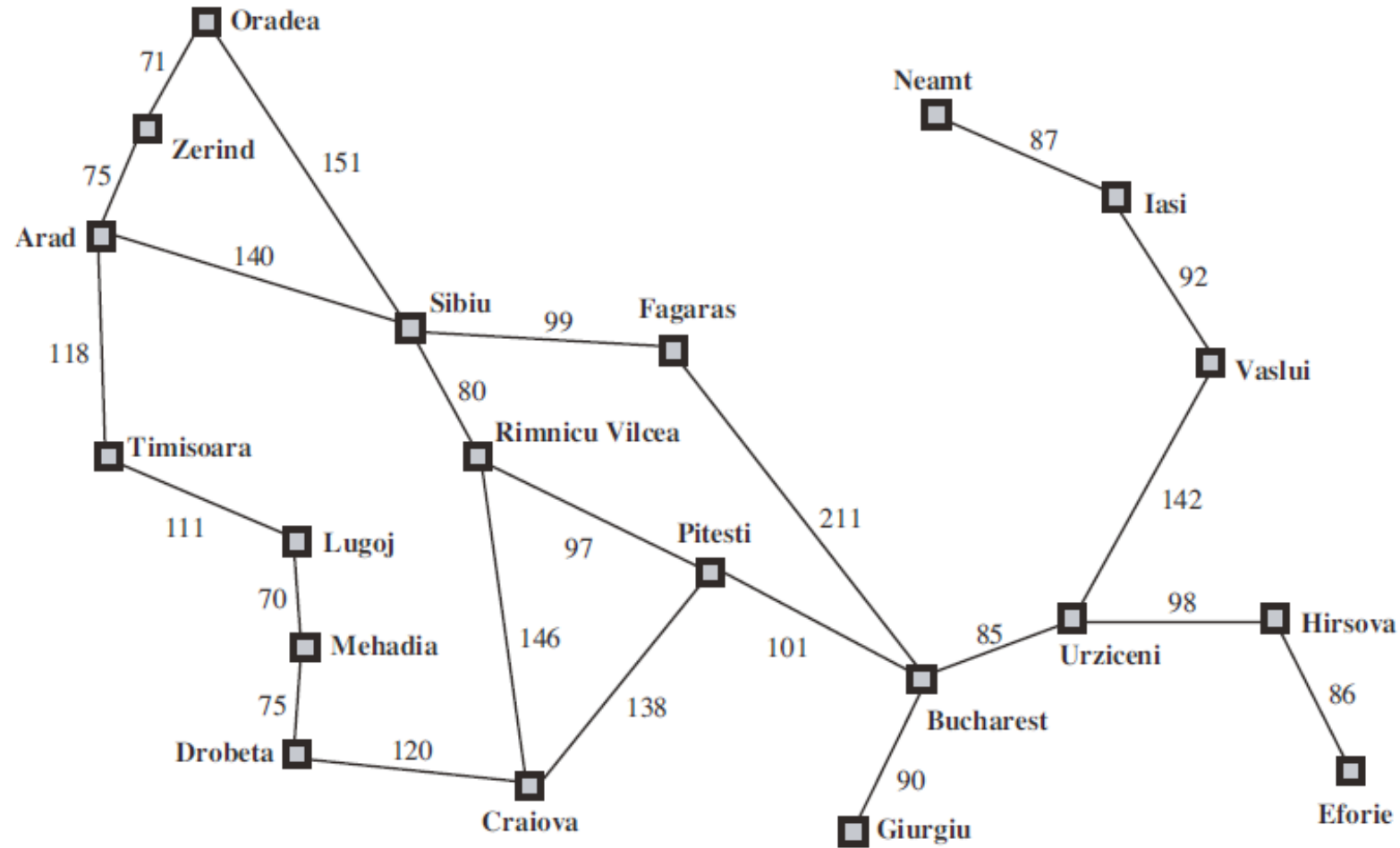


Problem Solving Agents

CHAPTER 3

Example

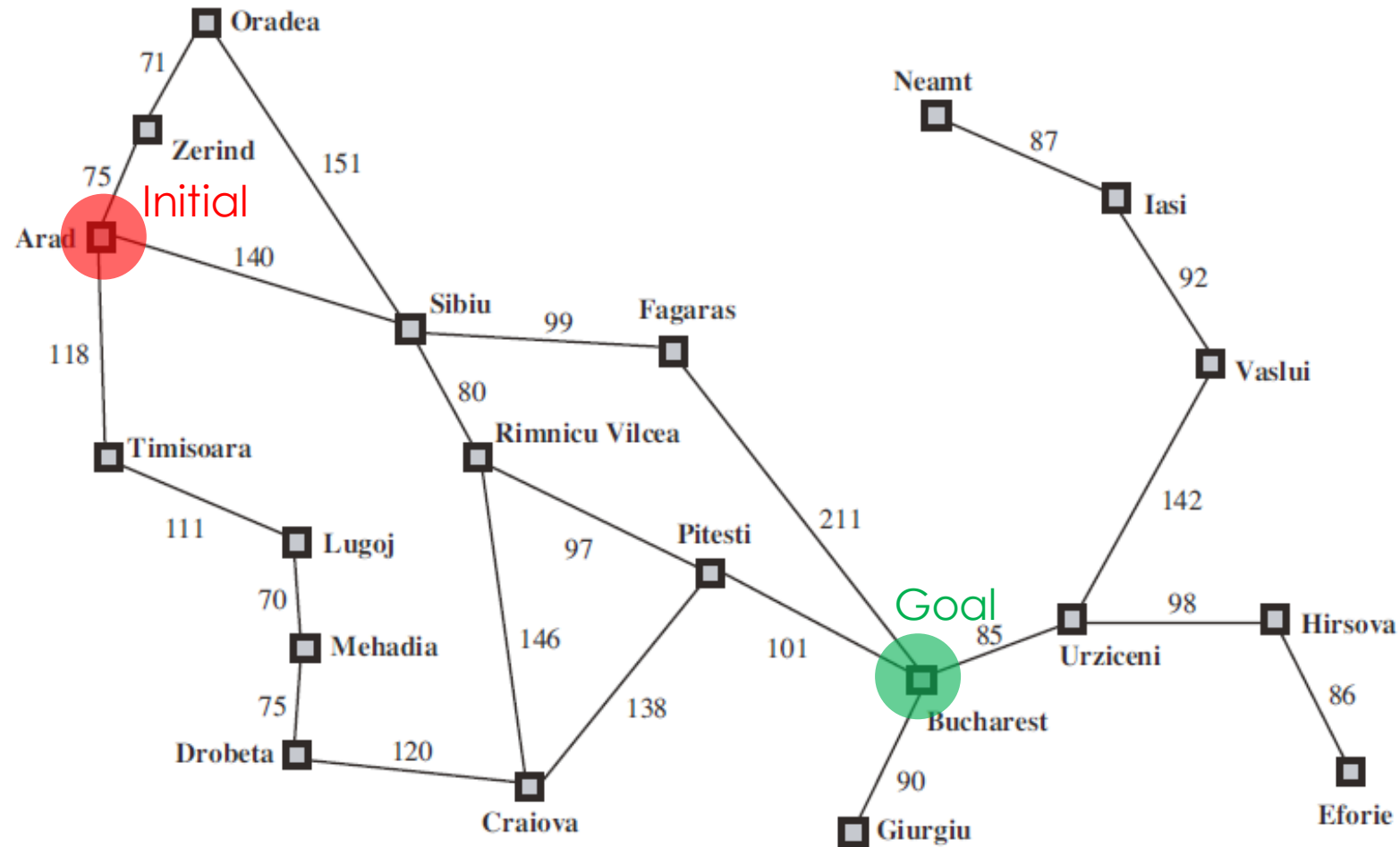


Problem Solving Agents

□ Goal Formulation

- Organize behavior of the agent
- Goal – set of states in the world where the goal is satisfied

Example



Problem Solving Agents

□ Goal Formulation

- Organize behavior of the agent
- Goal – set of states in the world where the goal is satisfied

□ Problem Formulation

- What are the actions?
- What are the states?

Assumptions about the Task Environment

- ☐ Observable or partially observable?
- ☐ Discrete or Continuous?
- ☐ Deterministic or Stochastic?
- ☐ Static or Dynamic?
- ☐ Episodic or Sequential?
- ☐ Multiple or Single Agent?

Assumptions about the Task Environment

- ☐ Observable or partially observable?
- ☐ Discrete or Continuous?
- ☐ Deterministic or Stochastic?
- ☐ Static or Dynamic?
- ☐ Episodic or Sequential?
- ☐ Multiple or Single Agent?

What is the solution to this problem?

Assumptions about the Task Environment

- ☐ Observable or partially observable?
- ☐ Discrete or Continuous?
- ☐ Deterministic or Stochastic?
- ☐ Static or Dynamic?
- ☐ Episodic or Sequential?
- ☐ Multiple or Single Agent?

What is the solution to this problem?

Finding a sequence of actions - Search!

Problem Solving Agents

□ Goal Formulation

- Organize behavior of the agent
- Goal – set of states in the world where the goal is satisfied

□ Problem Formulation

- What are the actions?
- What are the states?

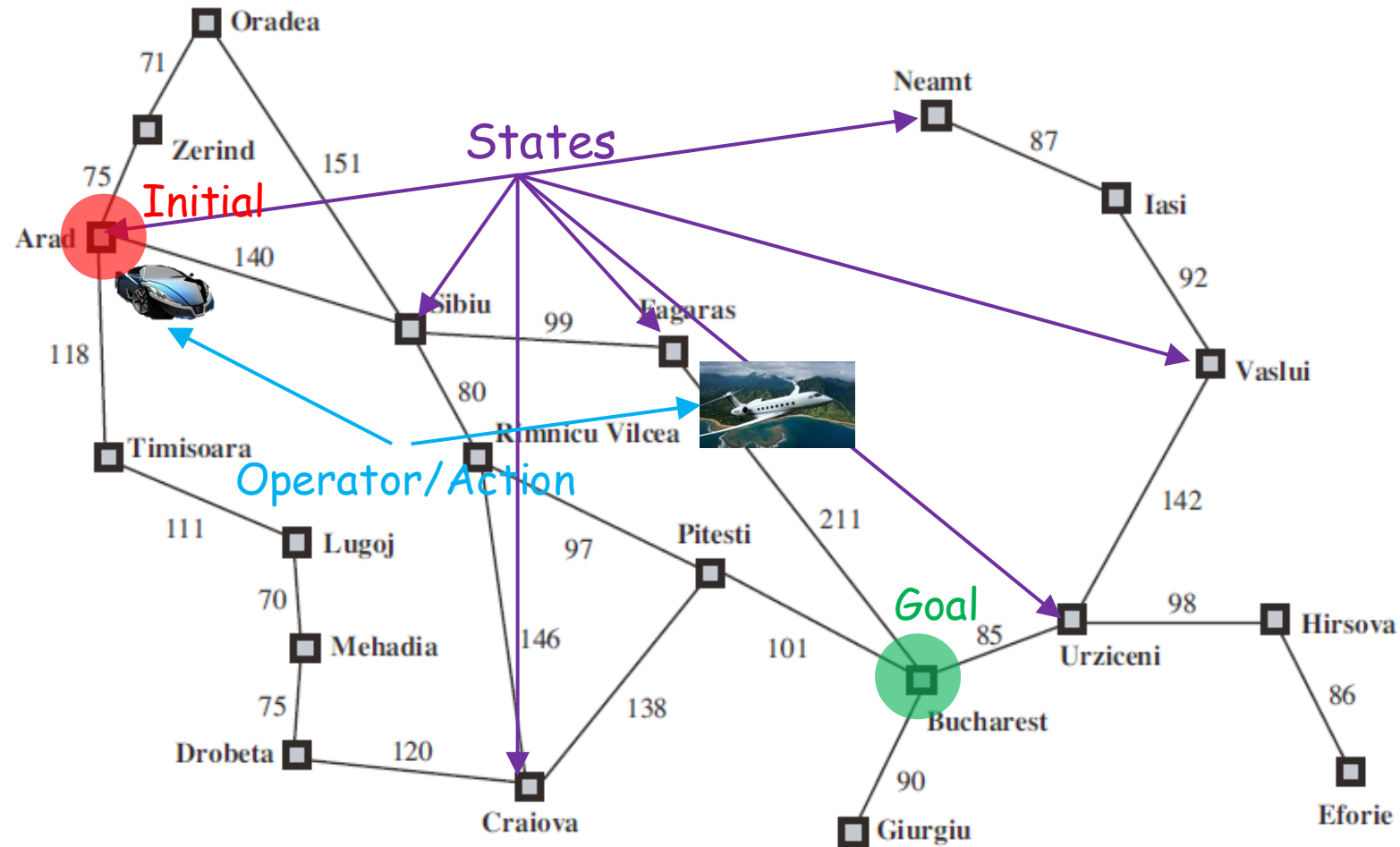
□ Search

- Finding the sequence of actions

□ Atomic Agents

- States are indivisible
- Searching through the states to reach the goal.

Example



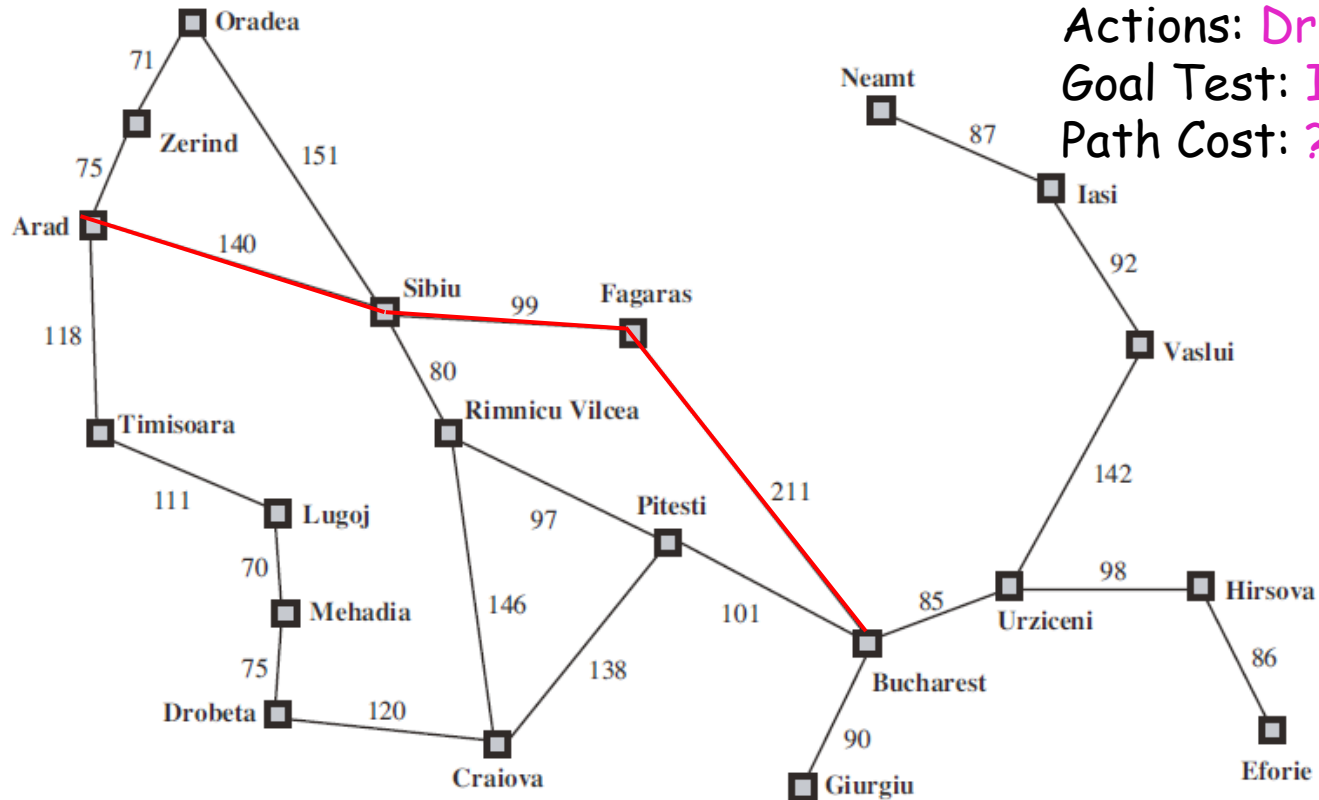
Single State Problem Formulation

❑ Problem can be defined by 5 components

1. Initial State: the state the agent starts
2. Actions: the set of operators that can be executed at a state
3. Transition model: returns the state that results from doing an action in a state
4. Goal test: determines whether a given state is a goal state
5. Path Cost: function that assigns a numeric cost to a path
 - Step cost: cost of taking a single action

- 
- State Space
 - Graph
 - Path

Example



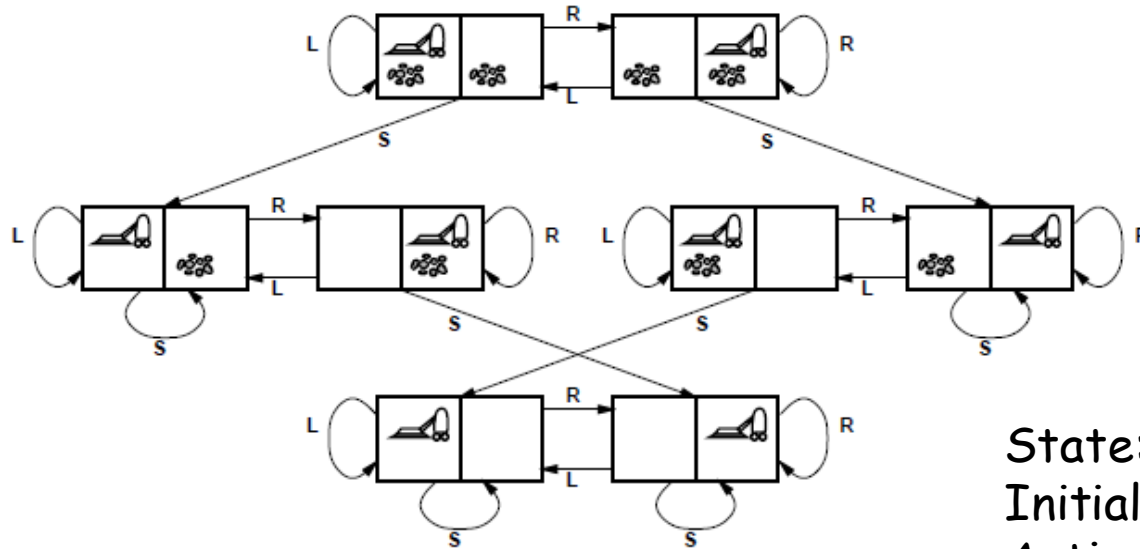
Initial State: Arad

Actions: Drive(Sibiu), Drive(Timisora)

Goal Test: In(Bucharest)

Path Cost: ?

Example: Toy Vacuum Problem



State: Robot and Dirt Locations

Initial State: Any State

Actions: Left, Right Suck

Goal Test: No Dirt

Path Cost: cost 1 per action?

Example: Eight Puzzle Problem

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

State: Tile Locations

Initial State: A specific tile configuration

Actions: move the blank tile left, right, up or down

Goal Test: tiles are in the required configuration

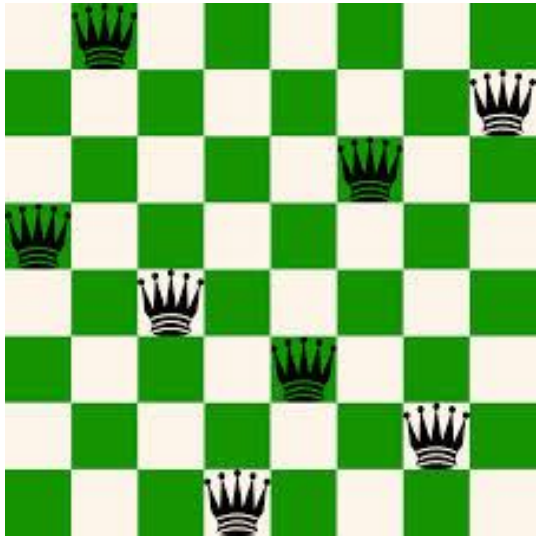
Path Cost: cost 1 per move?

8 puzzle - 181440 states

15 puzzle - 1.3 trillion states

24 puzzle - 10^{25} states

Example: 8 Queens Problem



State: Configuration of the Queens

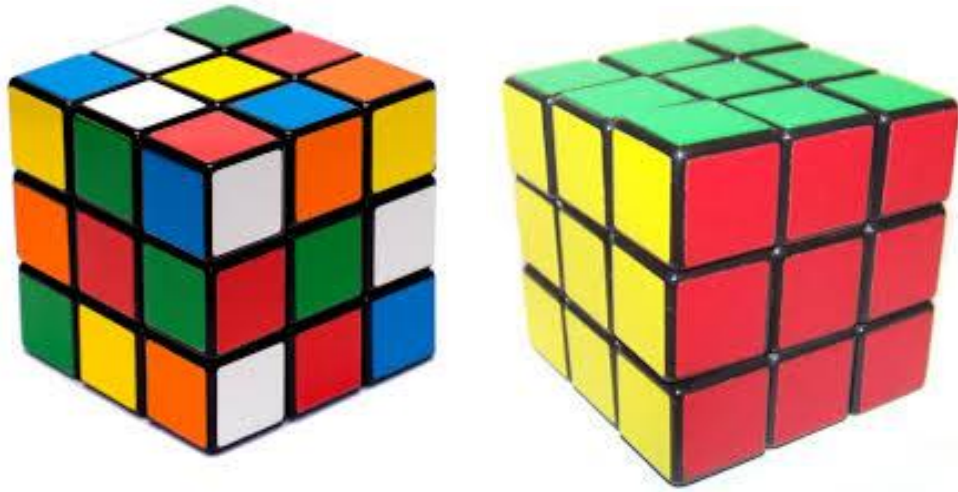
Initial State: Empty board

Actions: Add a queen to the board

Goal Test: configuration with 8 queens on the board with none attacking another

Path Cost: time taken to solve?

Example: Rubik's Cube



State: List of colors on each face

Initial State: A specific color pattern

Actions: rotate a row or column or a face

Goal Test: configuration has the same color on all tiles on every face

Path Cost: cost 1 per move?

Uninformed Search

Search - Trees

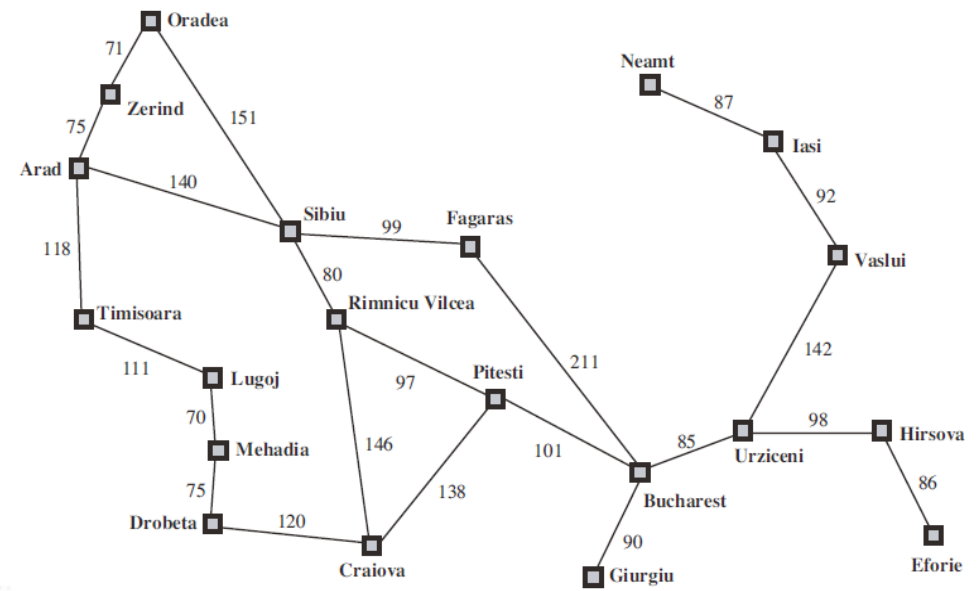
□ Basic Principle:

- Offline simulated exploration of search space
- Generate successors of already explored states

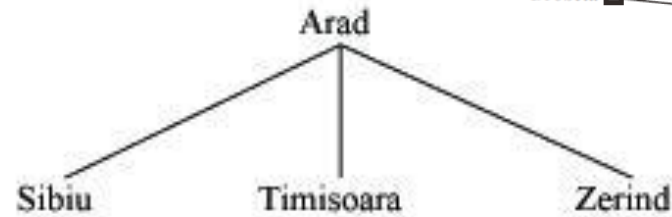
```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Example

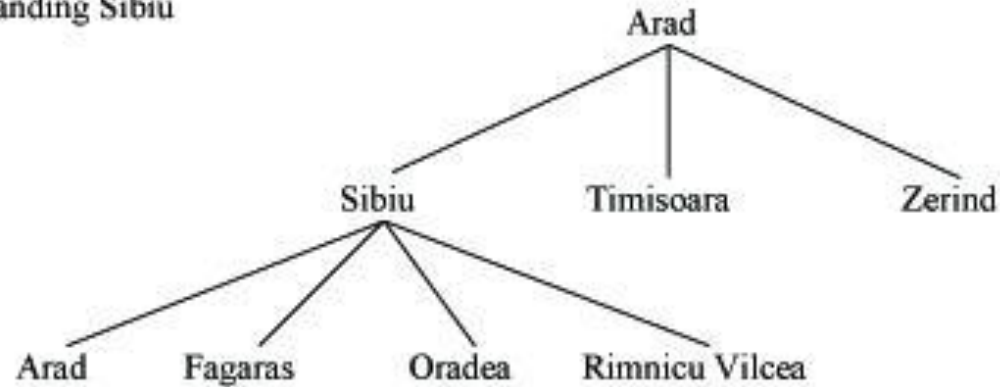
(a) The initial state



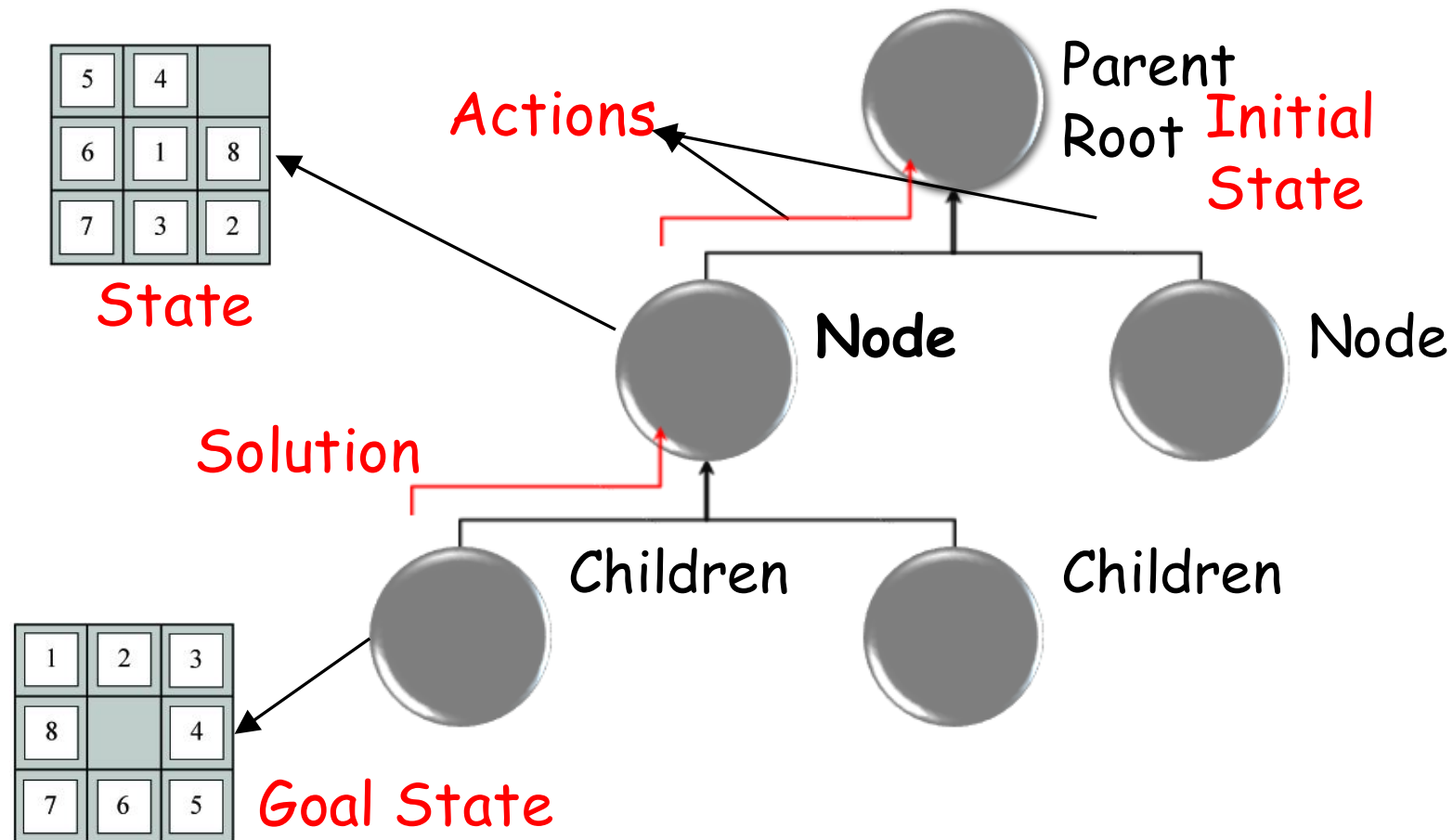
(b) After expanding Arad



(c) After expanding Sibiu



Search Space as a Tree



Graph Search

```
function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set
```

Search Strategies

❑ Strategies vary in the order in which nodes are picked for expansion

❑ Evaluating search strategies

- **Completeness** – Does it always find a solution if one exists?
- **Optimality** – Does it always find a least cost solution?
- **Space complexity** – How much memory is needed to perform search?
- **Time complexity** – How long does it take to find a solution?

❑ Time and Space complexities are measured

- b – maximum branching factor of the search tree
- d – shallowest depth of the least cost solution
- m – maximum depth of the search space

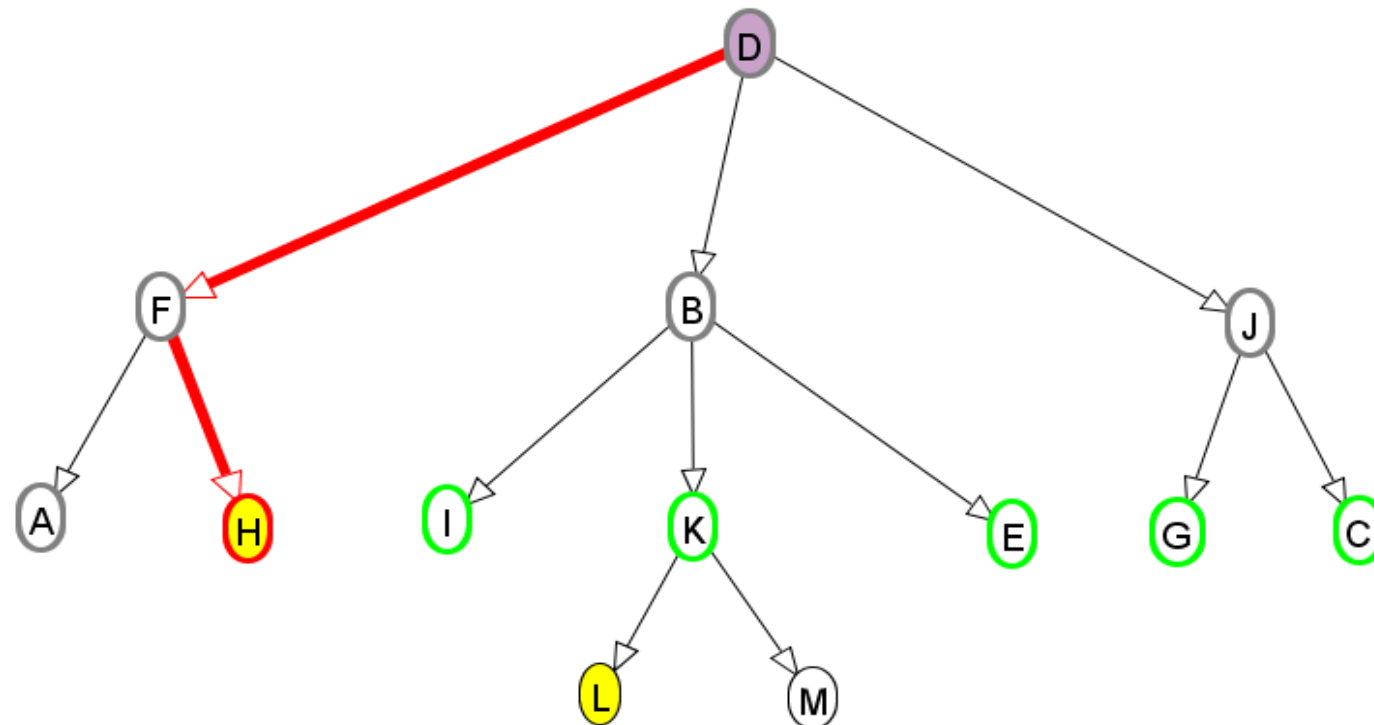
Uninformed search strategies

❑ Use only the information available in the problem definition

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

Breadth-first search (BFS)

- ❑ Expand shallowest unexpanded node
- ❑ **Implementation:** FIFO Queue; successors at the end of the queue



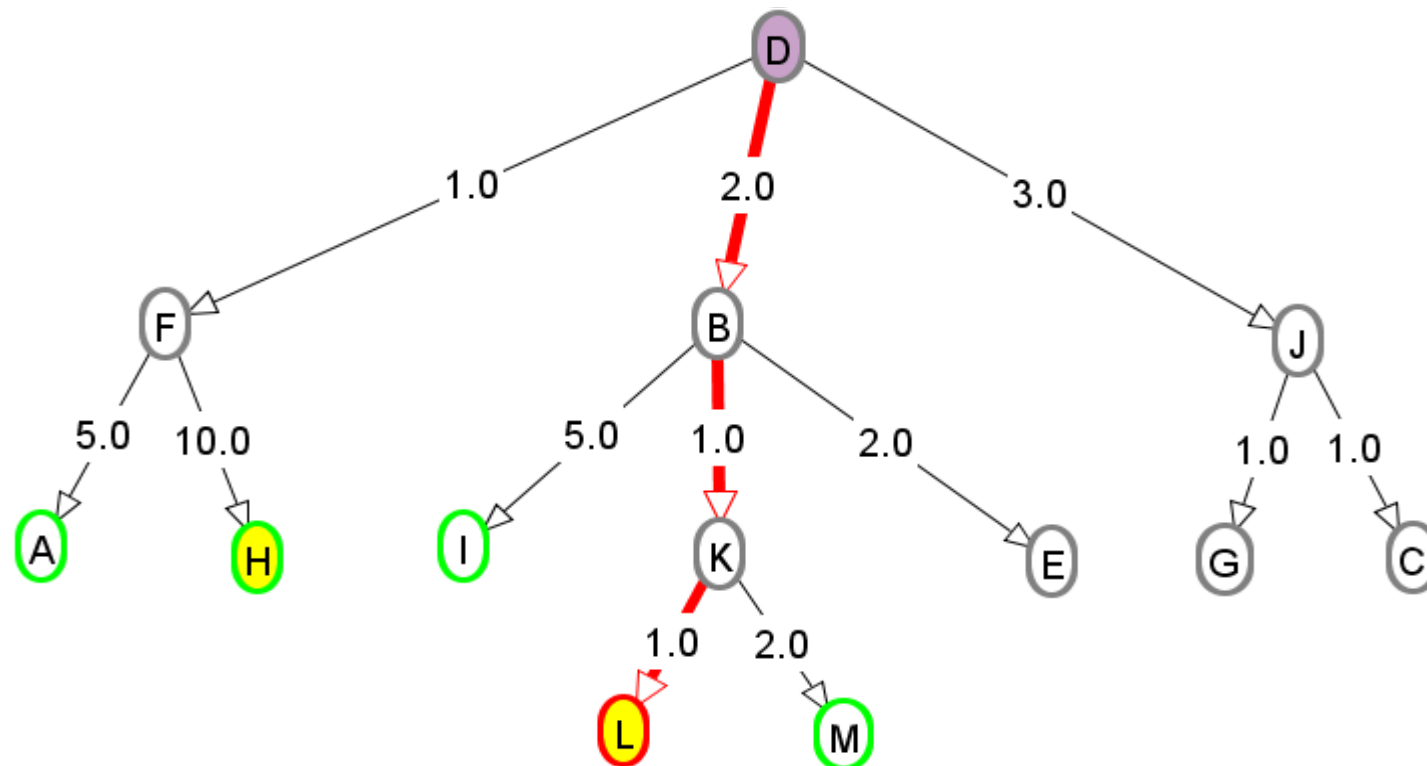
BFS – Analysis

- ❑ Completeness: Yes (if b is finite)
- ❑ Optimality: Not optimal; Yes- Uniform cost edges
- ❑ Time Complexity: exponential in $d+1$
- ❑ Space Complexity: exponential in $d+1$

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Uniform cost search (UCS)

- ❑ Expand least-cost ($g(n)$) unexpanded node
- ❑ **Implementation:** Priority queue – sort the nodes in the queue based on cost.



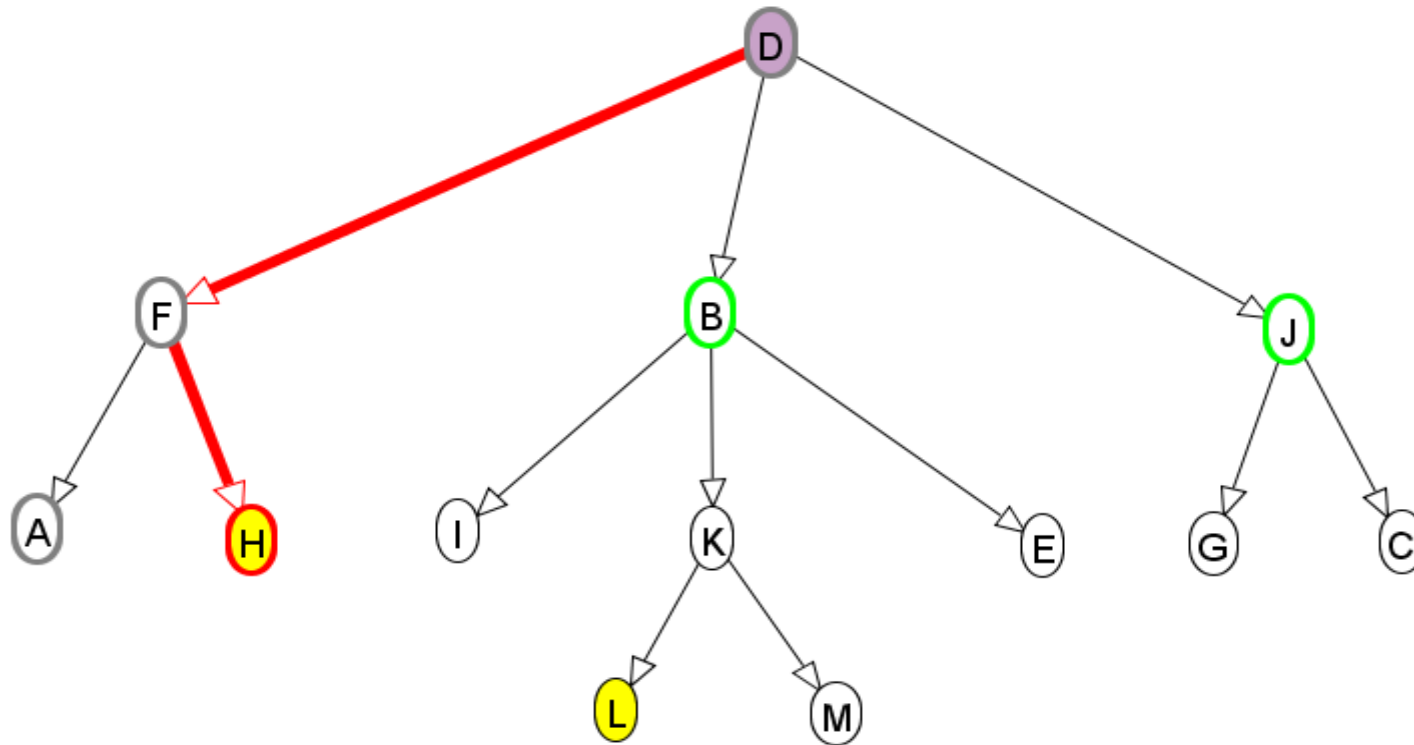
UCS - Analysis

- ❑ Completeness: Yes; if step cost $\geq \epsilon$
- ❑ Optimality: Yes; nodes are expanded in increasing order of $g(n)$
- ❑ Time Complexity: # of nodes with $g \leq \text{cost of optimal solution}(C^*)$ - $O(b^{\lceil C^*/\epsilon \rceil})$
- ❑ Space Complexity: # of nodes with $g \leq \text{cost of optimal solution}$ - $O(b^{\lceil C^*/\epsilon \rceil})$

Large subtrees with inexpensive steps may be explored before useful paths with costly steps

Depth-first search (DFS)

- ❑ Expand deepest unexpanded node
- ❑ **Implementation:** LIFO queue; successors at the front of the queue



DFS - Analysis

- ❑ Completeness: complete only in finite spaces; incomplete when there are loops and infinite spaces
- ❑ Optimality: No
- ❑ Time Complexity: $O(b^m)$; terrible when $m \gg d$; might be faster than BFS, when solutions are dense.
- ❑ Space Complexity: $1 + b + b + \dots + (m^{th} level)b = O(bm)$; Linear space!!

Depth	# nodes	Memory BFS	Memory DFS
16		10Eb	156Kb

Depth-limited search (DLS)

- ❑ Depth-first search with depth limit l
- ❑ **Implementation**: nodes at depth l have no successors.
- ❑ Only finite space to be explored.
- ❑ Completeness: Yes/No???
- ❑ Optimality: Yes/No???

Iterative deepening search(IDS)

function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** a solution

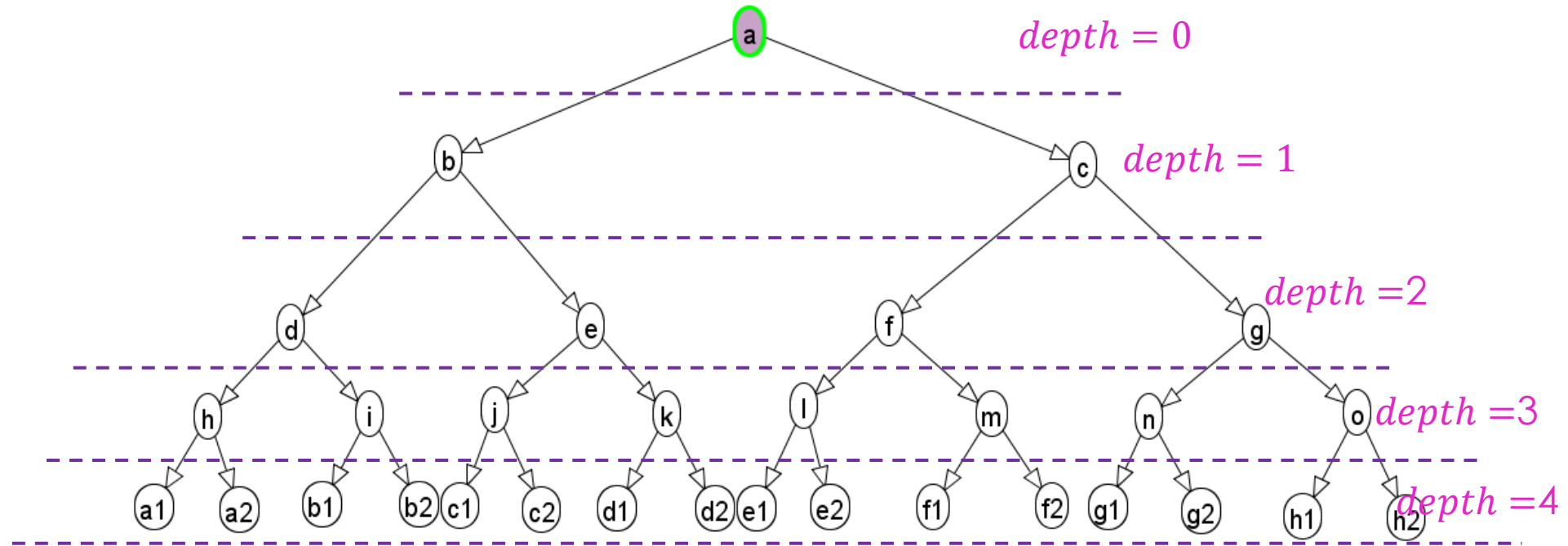
inputs: *problem*, a problem

for *depth* $\leftarrow$ 0 to ∞ **do**

result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)

if *result* $\neq$ cutoff **then return** *result*

end



IDS- Analysis

❑ Completeness: Yes!

❑ Optimality: Yes for Uniform cost edges, can be modified for non-uniform cost trees

- Iterative lengthening search

❑ Time Complexity: exponential in d

❑ Space Complexity: bd

- Let $b = 10$ and $d = 5$

- $N(IDS) = 50 + 400 + 3000 + 20000 + 10000 = 123450$

- $N(BFS) = 10 + 100 + 1000 + 10000 + 10000 = 111110$

- In general

- $N(IDS) = (d)b + (d-1)b^2 + \dots + (1)b^d \sim O(b^d)$

Summary

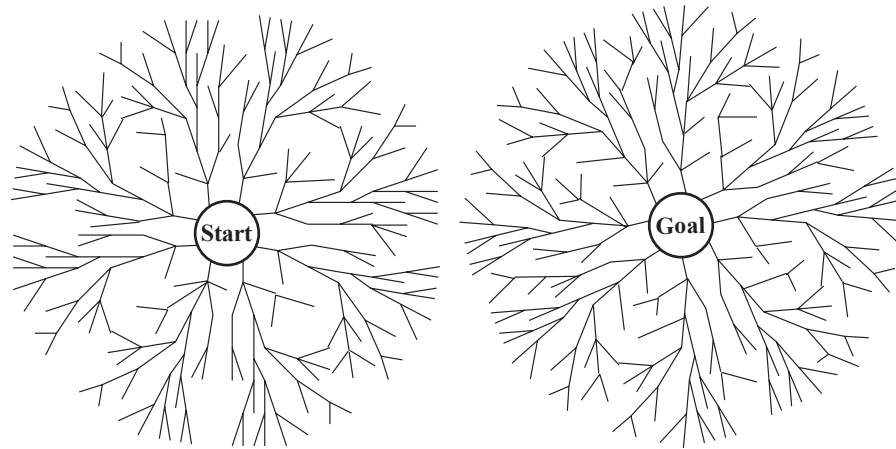
Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	bm	bl	bd
Optimal?	Yes*	Yes	No	No	Yes*

Bi-directional Search

- ❑ Run two simultaneous searches

- Forward search from the initial state
- Backward search from the goal state

- ❑ Replace the goal test function with a check to see whether the frontiers of the two searches intersect.

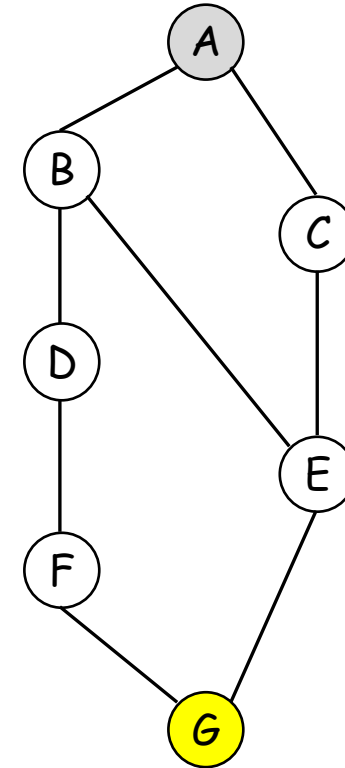


Tree/Graph Search

☐ BFS-?

☐ DFS-?

☐ IDDFS-?



Tree/Graph Search

☐ BFS-?

☐ DFS-?

☐ IDDFS-?

